

ENTERPRISE HUMAN RESOURCE MANAGEMENT SYSTEM

HRMS System Documentation

Complete Architecture, Data Models, Workflows, APIs, and Operational Reference for the Entire Application Suite



APPLICATION VERSION

v2.0.0 Enterprise

PUBLISHED DATE

September 04, 2026

CORE STACK

Django 5.x / PostgreSQL 16

SCOPE

30+ Modules Covered

ARCHITECTURE

Docker-First / Modular

REPOSITORY

shawn-cse/hrms

Table of Contents

1. System Overview & Architectural Blueprint	2. Core Architecture, Multi-Tenancy & Base Configuration (`base`)
3. Employee Profile & Lifecycle Management (`employee`)	4. Attendance & Time Tracking Engine (`attendance`)
5. Biometric Devices, Facial Recognition & Geofencing (`biometric`, `facedetection`, `geofencing`)	6. Leave & Absence Management (`leave`)
7. Payroll, Compensation & Benefits Administration (`payroll`)	8. Performance Management System (`pms`)
9. Asset & Equipment Lifecycle Management (`asset`)	10. Recruitment & Applicant Tracking System (`recruitment`)
11. Employee Onboarding Management (`onboarding`)	12. Employee Offboarding & Exit Clearance (`offboarding`)
13. Project Management & Timesheets (`project`)	14. Helpdesk & Internal Support Ticketing (`helpdesk`)
15. Reporting & Business Intelligence Engine (`report`)	16. RESTful API Architecture & Interactive Swagger UI (`hrms_api`)
17. Enterprise Authentication, RBAC & Security (`hrms_auth`, `base`)	18. Audit Trails & Regulatory Compliance (`hrms_audit`)
19. Automations, Notifications & Scheduled Tasks (`hrms_automations`, `notifications`)	20. Document Generation & Database Templates (`hrms_documents`, `hrms_dbtemplate`)
21. Database Backup & Disaster Recovery (`hrms_backup`, `pg_backup`)	22. External Enterprise Integrations (WhatsApp, LDAP, Outlook, Meet)
23. UI/UX, Dynamic Themes, Guided Tours & Accessibility	24. DevOps, Containerization & Production Deployment

CHAPTER 1

System Overview & Architectural Blueprint

High-level architecture, technology stack, directory structure, and modular design patterns.

1.1 Executive Summary

HRMS is an enterprise-grade, Docker-first Human Resource Management System built on Python 3.12, Django 5.x, PostgreSQL 16, Redis 7, Gunicorn, and Nginx. The system provides an end-to-end digital workplace platform governing every stage of the human capital lifecycle—from talent requisition, applicant tracking, and onboarding, to daily attendance, biometric synchronization, multi-tier leave workflows, automated payroll execution, performance appraisals (PMS), asset custody, project task tracking, internal helpdesk support, dynamic document generation, and formal offboarding clearance.

Designed around principles of modularity, data security, and multi-tenant organizational hierarchy, HRMS features fine-grained Role-Based Access Control (RBAC), multi-company isolation, two-factor authentication (2FA), continuous audit logging, event-driven email automations, scheduled background tasks, and a full-featured REST API with interactive Swagger/OpenAPI documentation.

1.2 Technology Stack

Layer	Technology	Version	Key Responsibilities
Core Framework	Django / Python	5.x / 3.12+	Web application framework, ORM, security middleware, authentication
Primary Database	PostgreSQL	16.x	Relational database, ACID transactions, relational integrity, JSONB fields
Cache & In-Memory	Redis	7.x	Session caching, application state, rate-limiting, message broker
Application Server	Gunicorn	21.x+	Production WSGI HTTP server with multi-worker concurrent request execution
Reverse Proxy / SSL	Nginx	Stable / Alpine	Reverse proxy, static asset caching, SSL/TLS termination, request buffering
Background Scheduler	Django APScheduler	0.3.0+	Cron tasks, automated payroll runs, daily attendance summaries, email queues
REST API Engine	Django REST Framework (DRF)	3.14+	RESTful API endpoints, SimpleJWT authentication, Swagger UI (drf_yasg)
Audit & History	Django Auditlog / Simple History	Current	Comprehensive changelog tracking, user action audits, field-level diffs
Containerization	Docker & Docker Compose	v2+	Multi-stage container builds, reproducible development and production stacks
Continuous Integration	GitHub Actions	Standard	Automated linting, test validation, security checks, Docker packaging

1.3 Architectural Design Patterns

1. Pluggable Dynamic App Discovery

HRMS implements a modular, pluggable application architecture. Rather than hardcoding dozens of URL includes inside the central `hrms/urls.py`, individual domain apps register their own routing and metadata dynamically via the Django `AppConfig.ready()` lifecycle hook:

```
# Example from leave/apps.py
class LeaveConfig(AppConfig):
    name = "leave"

    def ready(self):
        from django.urls import include, path
        from hrms.urls import urlpatterns

        settings.APPS.append("leave")
        urlpatterns.append(
            path("leave/", include("leave.urls")),
        )
        super().ready()
```

This pattern ensures that modules can be cleanly activated, deactivated, or decoupled for white-label or customer-specific deployments without modifying the core project engine.

2. Multi-Company Organizational Hierarchy

At the foundation of the data architecture lies the `base.Company` model. Organizational components (Departments, Job Positions, Work Types, Shifts, and Policies) and operational records (Employees, Attendances, Leave Requests, and Payslips) are mapped either directly or transitively to a Company entity. Custom middleware (`base.middleware.CompanyMiddleware`) resolves the active company context from the user's session or request headers, ensuring strict data isolation across multi-entity corporations.

3. Unified Request-and-Approve Engine

Workflows requiring managerial oversight—including Leave Requests, Attendance Regularizations, Shift Changes, Work Type Modifications, Reimbursements, and Asset Requests—route through a standardized `MultipleApprovalCondition` engine. Organizations can construct complex approval hierarchies (e.g., Immediate Supervisor -> Department Head -> HR Manager -> Finance Director) with custom conditions based on duration, expense value, or organizational rank.

1.4 Directory Layout

The repository is structured into modular domain packages, operational directories, and container configurations:

```

d:\2026__LO\Coding\hrms\
├─ hrms/                # Core Django project configuration, settings, base URLs, WSGI/ASGI
├─ base/                # Master organizational hierarchy, shifts, work types, companies, ESS
├─ employee/            # Employee profiles, bank info, emergency contacts, policies, notes
├─ attendance/          # Clock in/out, late/early calculation, overtime, work records
├─ biometric/           # Hardware integration (ZKTeco, Matrix COSEC, biometric devices)
├─ facedetection/       # Facial recognition attendance capture and verification
├─ geofencing/          # GPS location geofencing boundaries and perimeter validation
├─ leave/               # Leave types, accrual rules, requests, multi-tier approvals, holidays
├─ payroll/             # Salary contracts, allowances, deductions, tax slabs, batch payslips
├─ pms/                 # Performance Management System: OKRs, KPIs, 360 feedback, reviews
├─ asset/               # Hardware/software inventory, asset allocation, return condition photos
├─ recruitment/         # Requisitions, pipeline stages, resume parsing, interview scoring
├─ onboarding/          # Pre-boarding portal, document collection, departmental checklists
├─ offboarding/         # Resignations, clearance workflows (IT, HR, Finance), exit interviews
├─ project/             # Projects, milestones, Kanban task boards, billable timesheets
├─ helpdesk/            # Internal support ticketing, category routing, SLA tracking, FAQs
├─ report/              # BI reporting engine, custom filters, scheduled exports (CSV/Excel/PDF)
├─ hrms_api/            # RESTful APIs, SimpleJWT auth, Swagger OpenAPI documentation
├─ hrms_auth/           # Custom HRMSUser model, authentication backends, permission groups
├─ hrms_audit/          # Audit trail logging, field diff tracking, account lock/unlock history
├─ hrms_automations/    # Event triggers, scheduled cron jobs, automated mail actions
├─ notifications/       # In-app notification drawer, broadcast announcements, email dispatches
├─ hrms_documents/      # Dynamic document generation, certificate requests, PDF outputs
├─ hrms_dbtemplate/     # Database-driven rich text templates with dynamic merge tags
├─ hrms_backup/         # PostgreSQL automated backups, local archives, Google Drive sync
├─ hrms_ldap/           # Enterprise Active Directory / OpenLDAP directory synchronization
├─ hrms_meet/           # Google Meet video conference room integration for interviews
├─ hrms_theme/          # Dynamic white-label CSS theme engine and branding customizer
├─ hrms_tour/           # Interactive guided UI product tours (tourController.js)
├─ hrms_views/          # Dynamic view filters, saved queries, toggleable table columns
├─ hrms_widgets/        # Dashboard widget framework and metrics visualizers
├─ whatsapp/            # Meta WhatsApp Business API integration for alerts and messaging
├─ dynamic_fields/      # Runtime custom fields on models without database migrations
├─ accessibility/       # WCAG compliance features: high contrast, dyslexic font, text resizing
├─ static/              # Source static assets (JavaScript, CSS, vendor libraries, icons)
├─ staticfiles/         # Collected static files for production deployment via WhiteNoise/Nginx
├─ templates/           # Global HTML templates, authentication views, base master layouts
├─ docker/              # Dockerfile, Nginx configurations, container entrypoint scripts
└─ docs/                # System documentation, HTML print assets, generated executive PDF

```

CHAPTER 2

Core Architecture, Multi-Tenancy & Base Configuration (`base`)

Organizational structure, multi-company hierarchy, shifts, work types, approval chains, and system middlewares.

2.1 Overview & Responsibilities

The `base` application represents the architectural spine of the HRMS platform. It houses the foundational data models and services that dictate company structure, organizational hierarchies, operational working shifts, rotating schedules, holiday calendars, public announcements, and the unified request-and-approval engine.

In addition to core organizational data, `base` implements mission-critical system middlewares enforcing tenant scoping, password rotation, two-factor authentication, and HTTP request sanitization.

2.2 Database Models Specification

The `base` module defines over 45 relational models governing corporate operations:

Model Name	Purpose	Primary Attributes & Relationships
Company	Root multi-tenancy entity representing legal corporate bodies.	company_name , registration_number , email , phone , website , address , country , currency , logo , timezone , is_active
CompanyGroupAssignment	Assigns security permission groups and roles on a per-company basis.	company (FK), group (FK to auth.Group), assigned_by
Department	Functional divisions within a company.	department_name , company (FK), department_manager (FK to Employee), parent_department (self-referential FK for sub-departments)
JobPosition	Standardized job titles and organizational roles.	job_position_name , department (FK), company (FK), job_description , maximum_headcount
JobRole	Granular responsibilities, duties, and qualifications tied to job positions.	job_role_name , job_position (FK), responsibilities , qualifications
WorkType	Core work arrangements (e.g. Regular Office, Remote, Hybrid, Flexi).	work_type_name , company (FK), is_remote , requires_approval
RotatingWorkType	Multi-week rotating work arrangements (e.g. 2 weeks on-site, 1 week remote).	name , company (FK), cycle_frequency_days , is_active
RotatingWorkTypeAssign	Assignment of rotating work schedules to individual employees.	employee (FK), rotating_work_type (FK), start_date , end_date
EmployeeType	Employment classifications (e.g. Permanent, Probationary, Contract, Intern).	employee_type_name , company (FK), benefits_eligible
EmployeeShift	Standard operational work timings for employees.	shift_name , company (FK), start_time , end_time , grace_time_minutes , break_duration_minutes , working_days
EmployeeShiftSchedule	Daily schedule exceptions and specific shift assignments per date.	employee (FK), shift (FK), schedule_date , notes
RotatingShift	Recurring rotational shift patterns (e.g. Morning -> Evening -> Night).	name , company (FK), rotation_frequency_days , shifts (M2M)
RotatingShiftAssign	Links an employee to an automated recurring shift rotation.	employee (FK), rotating_shift (FK), effective_from , active

Model Name	Purpose	Primary Attributes & Relationships
Roster	Master published weekly/monthly duty roster for staff allocations.	employee (FK), shift (FK), date , published_by , status
RosterPublishLog	Audit log tracking when rosters are published and notices dispatched.	roster (FK), published_at , published_by , recipients_count
WorkTypeRequest	Employee self-service request to alter work type arrangement.	employee (FK), requested_work_type (FK), start_date , end_date , reason , status
ShiftRequest	Employee self-service request for temporary shift swap or change.	employee (FK), requested_shift (FK), effective_date , reason , status
MultipleApprovalCondition	Configurable approval chains based on roles, departments, or limits.	workflow_type , step_number , approver_role , approver_user (FK), is_mandatory
Holidays	Official calendar of national, regional, and company holidays.	holiday_name , holiday_date , company (FK), is_recurring , description
CompanyLeaves	Mandatory company shutdown periods or seasonal collective leaves.	name , start_date , end_date , company (FK), deduct_from_leave_balance
HRSMailTemplate	Reusable email notification templates with dynamic placeholder variables.	title , subject , body_html , mail_type , available_variables
MailLog	Historical transmission log of all outbound emails dispatched by the system.	template (FK), recipient_email , subject , sent_at , delivery_status , error_message
Penalty	Disciplinary penalty rules for chronic late arrivals or unauthorized absence.	penalty_name , violation_threshold_count , action_type , deduction_amount

2.3 Core Middlewares & System Interceptors

HRMS enforces critical security, tenant isolation, and session integrity policies via custom Django middlewares defined in `base.middleware` and `hrms.hrms_middlewares`:

1. **CompanyMiddleware** : Intercepts every incoming HTTP request. Identifies the authenticated user's assigned company context or reads the explicit company header (`x-Company-ID`). Attaches `request.company` and activates database filtering to isolate tenant data.
2. **ForcePasswordChangeMiddleware** : Inspects user profile state upon successful authentication. If `user.force_password_change` is set (such as following account provisioning or admin reset), the user is forcibly redirected to the password reset view, blocking all other endpoints until a secure password is established.

3. **TwoFactorAuthMiddleware** : Enforces Time-Based One-Time Password (TOTP) verification for privileged user tiers (System Administrators, HR Directors, Payroll Managers) before allowing access to administrative or financial views.
 4. **SVGSecurityMiddleware** : Inspects all file uploads containing SVG images. Parses and sanitizes XML payloads to neutralize embedded `<script>`, `onload`, or malicious SVG-based Cross-Site Scripting (XSS) vectors.
 5. **DefaultLanguageMiddleware** : Detects user-selected language preferences from cookies or browser accept headers and activates Django's `i18n` locale context.
 6. **MethodNotAllowedMiddleware** & **MissingParameterMiddleware** : Intercepts ill-formed requests or unsupported HTTP verbs, transforming raw Django exceptions into standardized JSON responses with error codes.
-

2.4 User Interfaces & Dashboards

The `base` module provides several specialized UI layouts: - **Unified Main Dashboard (`/dashboard/`)**: Executive overview presenting KPI metrics: Total Active Headcount, Today's Attendance Rate, Leave Breakdown Donut Chart, Department Distribution Bar Chart, Gender Split, and Company Announcements. - **Employee Self-Service (ESS) Dashboard (`/ess/`)**: Employee-centric portal showing live check-in status, remaining leave balances, upcoming holidays, shift schedules, assigned tasks, and quick-action buttons to request leave or regularize attendance. - **Request & Approval Central**: Centralized queue where managers review, approve, or reject pending requests across Leave, Attendance, Shifts, Assets, and Expenses with audit notes.

CHAPTER 3

Employee Profile & Lifecycle Management (`employee`)

Employee master records, statutory information, banking, emergency contacts, disciplinary tracking, and company policies.

3.1 Overview & Responsibilities

The `employee` application manages the comprehensive digital profile of every staff member. It serves as the single source of truth for personal data, statutory identification, compensation structures, banking credentials, emergency contacts, academic qualifications, previous work history, company policies, and disciplinary proceedings.

3.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
Employee	Master employee profile linked 1:1 to the system HRMSUser .	badge_id , first_name , last_name , email , phone , gender , dob , marital_status , nationality , blood_group , avatar , joining_date , is_active
EmployeeWorkInformation	Employment details, contractual status, and reporting lines.	employee (1:1), company (FK), department (FK), job_position (FK), job_role (FK), shift (FK), work_type (FK), reporting_manager (FK), probation_end_date
EmployeeBankDetails	Payroll disbursement bank account details and statutory IDs.	employee (1:1), bank_name , account_number , branch_name , ifsc_or_swift_code , pan_or_ssn_number , tax_id
EmployeeTag	Metadata categorization tags for skills, projects, or committees.	tag_name , color_code , company (FK)
EmployeeNote	Confidential notes, HR interview logs, and performance remarks.	employee (FK), author (FK), note_title , description , is_confidential , created_at
NoteFiles	Document attachments associated with employee notes.	note (FK), attachment_file , file_name , uploaded_at
Policy	Corporate compliance policies, handbooks, and code of conduct.	policy_title , company (FK), description , effective_date , mandatory_acknowledgement
PolicyMultipleFile	PDF attachments and official documents linked to corporate policies.	policy (FK), document_file , uploaded_at
BonusPoint	Recognition and reward points granted by managers or peers.	employee (FK), points , awarded_by (FK), reason , date_awarded
Actiontype	Standardized disciplinary action categories (e.g. Warning, Suspension, Fine).	action_name , severity_level , company (FK)
DisciplinaryAction	Records of formal infractions, investigations, and penalties.	employee (FK), action_type (FK), infraction_date , description , decision , penalty_deduction , status
ProfileEditFeature	Fine-grained controls dictating which profile fields employees can edit.	field_name , allow_employee_edit , requires_hr_approval
EmployeeGeneralSetting	Global settings for employee auto-numbering, prefix, and defaults.	employee_id_prefix , next_auto_number , require_photo_on_creation

3.3 Lifecycle Operations & Workflows

1. Employee Creation & Onboarding Handoff

When a candidate successfully finishes onboarding or an HR admin manually creates a staff record: 1. An `Employee` profile is generated, and a unique `badge_id` is assigned according to `EmployeeGeneralSetting`. 2. A corresponding `HRMSUser` authentication record is provisioned with a secure random password and temporary flag. 3. Reporting manager, shift timings, work types, and departments are configured in `EmployeeWorkInformation`. 4. Mandatory corporate policies (`Policy`) are automatically bound to the new employee, triggering an acknowledgement notification on their next login.

2. Disciplinary Action & Governance

When compliance violations or behavioral infractions occur: 1. HR initiates a `DisciplinaryAction` linking the employee and an `Actiontype`. 2. Documentation, witness statements, and formal warnings are attached. 3. If financial penalties or suspensions are levied, `penalty_deduction` communicates directly with the `payroll` module during the subsequent monthly pay cycle to apply structured wage deductions.

3. Employee Self-Service (ESS) Profile Management

Employees can review and update non-critical personal data (e.g. personal contact numbers, emergency contacts, residential address) directly from their ESS portal. The `ProfileEditFeature` model allows HR administrators to lock sensitive fields (e.g. Bank Account Number, Legal Name, Date of Birth, Job Title) so that modifications require formal HR ticket submission and verification.

CHAPTER 4

Attendance & Time Tracking Engine (`attendance`)

Clock in/out tracking, grace periods, overtime computation, late-in/early-out rules, and work records.

4.1 Overview & Responsibilities

The `attendance` application records, computes, and audits employee working hours across physical offices, remote workstations, and field operations. It tracks precise timestamps, evaluates attendance against scheduled shift rules, calculates grace period tolerances, detects late arrivals and early departures, manages overtime approvals, and provides daily summary work records utilized directly by the payroll calculation engine.

4.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
Attendance	Primary daily attendance record per employee per calendar date.	employee (FK), attendance_date , attendance_clock_in , attendance_clock_out , attendance_clock_in_date , attendance_clock_out_date , minimum_hour , at_work , attendance_worked_hour , is_validated
AttendanceActivity	Raw punch logs capturing individual clock-in/out events and sources.	employee (FK), attendance_date , time , clock_in , clock_out , device_id , ip_address , latitude , longitude
BatchAttendance	Bulk attendance creation records for shifts, teams, or departments.	batch_name , company (FK), department (FK), shift (FK), date , created_by
AttendanceOverTime	Tracks verified overtime hours and managerial approval state.	employee (FK), attendance (FK), month_sequence , overtime_hour , status (Requested/Approved/Rejected), approved_by
AttendanceLateComeEarlyOut	Tracks minutes and frequency of late arrivals and early departures.	employee (FK), attendance (FK), attendance_date , type (Late Come / Early Out), duration_minutes , reason , is_excused
AttendanceValidationCondition	Logical threshold rules determining daily status (Present, Half Day, Absent).	company (FK), minimum_hours_full_day , minimum_hours_half_day , auto_validate
GraceTime	Configurable grace tolerance windows before late-arrival penalties trigger.	company (FK), shift (FK), allowed_late_minutes , allowed_grace_frequency_per_month
AttendanceGeneralSetting	Global operational rules for auto-clock-out, selfie capture, and IP locks.	auto_clock_out_time , enable_ip_restriction , require_selfie_on_punch , enable_overtime_approval
WorkRecords	Aggregated daily work units feeding directly into payroll processing.	employee (FK), date , regular_hours , overtime_hours , loss_of_pay_units , salary_ready
AttendanceConflictResolution	Resolution record for punch collisions, missing check-outs, or hardware drift.	attendance (FK), conflict_type , original_value , corrected_value , resolved_by

Model Name	Purpose	Primary Attributes & Relationships
AttendanceDailyHours & AttendanceSummaryHours	Pre-aggregated caches of daily and monthly hours for instant dashboard reporting.	employee (FK), period_date , total_hours_worked , overtime_total , late_count , early_out_count

4.3 Key Workflows & Business Logic

1. Clock In / Clock Out Lifecycle

1. When an employee punches via the web portal, mobile app, or biometric device, an AttendanceActivity record is logged with timestamp, device source, IP, and GPS coordinates.
2. If no open Attendance record exists for the date, a new record is initialized with attendance_clock_in .
3. Subsequent punches calculate total duration at work (attendance_worked_hour).
4. Shift start time is compared with actual check-in time. If the arrival exceeds the assigned shift's GraceTime , an AttendanceLateComeEarlyOut record is registered.
5. If total worked hours fall below minimum_hours_half_day , the day is marked as Absent or Half Day based on AttendanceValidationCondition .

2. Overtime & Regularization Requests

- When an employee works beyond their scheduled shift conclusion, surplus minutes are captured. If overtime tracking is active, an AttendanceOverTime request is routed to the employee's reporting manager.
- If an employee forgets to clock in/out, they submit an **Attendance Regularization Request** via ESS with justification notes. Upon manager approval, the master Attendance record updates and is marked is_validated = True .

CHAPTER 5

Biometric Devices, Facial Recognition & Geofencing (`biometric`, `facedetection`, `geofencing`)

Hardware time clock synchronization, AI camera face recognition, and GPS boundary validation.

5.1 Overview & Responsibilities

To accommodate varied corporate environments—from traditional corporate offices and manufacturing facilities to remote and field workforces—HRMS incorporates three advanced verification modules: 1. `biometric`: Direct hardware network integration with standalone biometric terminals (ZKTeco, Matrix COSEC, TCP/IP time-clocks). 2. `facedetection`: Browser and kiosk camera-based facial recognition attendance capture. 3. `geofencing`: GPS location perimeter validation for mobile and field employees.

5.2 Biometric Device Integration (`biometric`)

Database Models

- `BiometricDevices`: Represents physical hardware clocks. Attributes: `device_name`, `device_ip`, `port`, `device_type` (ZKTeco, Matrix, Suprema), `username`, `password`, `company` (FK), `last_sync_time`, `is_active`.
- `BiometricEmployees`: Maps HRMS internal `badge_id` to hardware device user IDs, fingerprint templates, and RFID card numbers.
- `COSECArrivalArguments`: Specialized parameters for Matrix COSEC enterprise device communication.

Synchronization Architecture

A background synchronization service connects via TCP socket or HTTP API to active biometric devices on scheduled cron intervals: 1. Queries the terminal's punch log buffer. 2. Extracts raw punch records (`user_id`, `timestamp`, `in_out_mode`). 3. Resolves the corresponding `Employee` record via `BiometricEmployees`. 4. Ingests raw punches into `AttendanceActivity` and computes daily `Attendance`. 5. Clears or acknowledges processed logs on the physical terminal to prevent duplicates.

5.3 Facial Recognition Attendance (`facedetection`)

Database Models & Workflow

- `FaceDetection`: System configuration for camera-based face verification, confidence threshold (e.g. 0.85), and anti-spoofing flags.
- `EmployeeFaceDetection`: Stores 128-dimensional facial feature vector embeddings extracted from employee photo uploads.

Kiosk & Web Punch Flow

1. An on-site tablet kiosk or employee browser activates the camera.
2. Captures face frame, evaluates liveness (blink or micro-movement detection to prevent photo spoofing).

3. Compares facial embedding vectors against the registered gallery in `EmployeeFaceDetection` using Euclidean distance / cosine similarity.
4. On positive match (confidence $\geq$ threshold), generates an automated clock-in/out in `attendance`.

5.4 Geofencing Boundary Validation (`geofencing`)

Database Models & Logic

- `GeoFencing` : Office or work-site boundary definition: `location_name` , `company` (FK), `latitude` , `longitude` , `radius_in_meters` , `is_active` .

Validation Workflow

When field employees clock in via mobile ESS: 1. The client sends HTML5 Geolocation coordinates (`lat` , `lng` , `accuracy`). 2. The server executes the **Haversine formula** against all active `GeoFencing` sites assigned to the employee:

$$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta \lambda}{2}\right)}\right)$$

3. If distance $d \leq \text{radius}$, the punch is approved and tagged with location. 4. If out of bounds, the punch is rejected or submitted as an exception flag requiring manager review.

CHAPTER 6

Leave & Absence Management (`leave`)

Leave types, accrual logic, available balances, multi-tier approvals, carry-overs, and holiday calendars.

6.1 Overview & Responsibilities

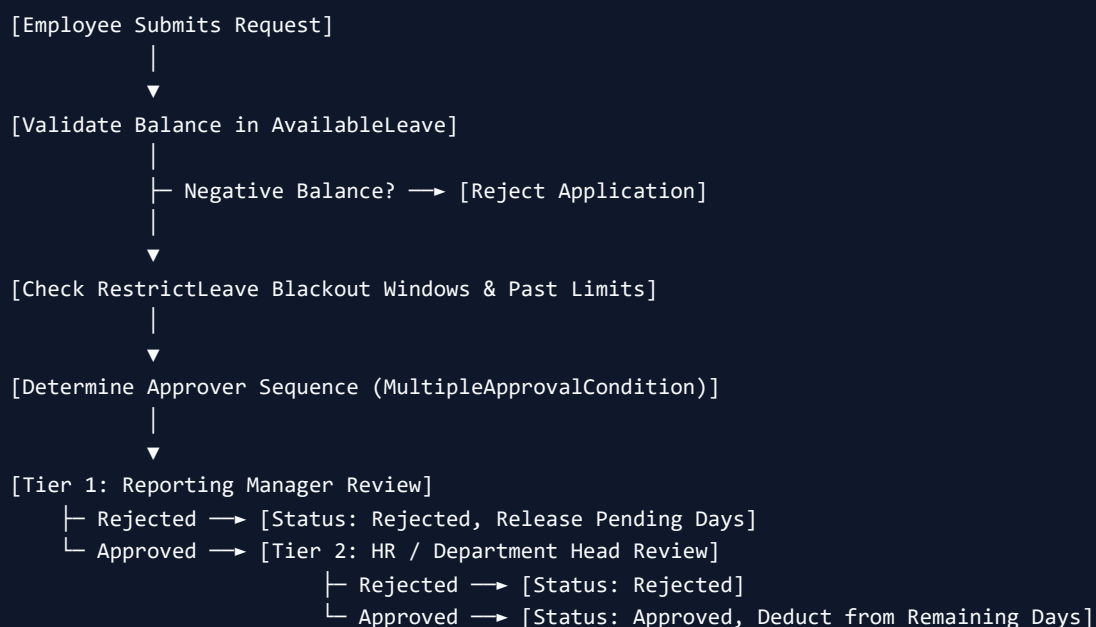
The `leave` application provides comprehensive absence administration. It manages categorized leave quotas (Annual, Sick, Casual, Maternity, Paternity, Compensatory Off, Unpaid Leave), calculates dynamic monthly or annual accruals, verifies supporting medical documents, enforces blackout dates, and manages multi-level approval hierarchies.

6.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
LeaveType	Definition of distinct leave categories.	name , company (FK), color_code , is_paid , is_carry_forward , max_carry_forward_days , is_encashable , requires_approval , requires_attachment
LeaveTypeCondition	Accrual rules, probation eligibility, and duration limits.	leave_type (FK), accrual_frequency (Monthly/Yearly/Lump Sum), probation_period_months , max_consecutive_days , sandwich_rule_active
AvailableLeave	Live balance ledger per employee per leave type.	employee (FK), leave_type (FK), allocated_days , used_days , pending_approval_days , carry_forward_days , remaining_days
LeaveRequest	Employee leave application record.	employee (FK), leave_type (FK), start_date , end_date , duration (Full Day / Half Day), reason , status (Requested/Approved/Rejected/Cancelled)
LeaverequestFile	File attachments (medical certificates, travel orders) for leave requests.	leave_request (FK), attachment_file , uploaded_at
LeaverequestComment	Two-way dialogue thread between applicant and approving managers.	leave_request (FK), commenter (FK to User), comment_text , created_at
LeaveAllocationRequest	Employee requests for additional leave grants (e.g. Compensatory Off).	employee (FK), leave_type (FK), requested_days , reason , status
LeaveallocationrequestComment	Discussion notes on manual leave allocation claims.	leave_allocation_request (FK), commenter (FK), comment_text
LeaveRequestConditionApproval	Tracks individual stage approvals in a multi-tier approval chain.	leave_request (FK), approval_stage , approver (FK), action (Approved/Rejected), action_date
RestrictLeave	Departmental or corporate blackout windows restricting leave.	company (FK), department (FK), start_date , end_date , reason
LeaveGeneralSetting	Global parameters governing retroactive leave limits and calendar displays.	max_past_days_to_apply , include_holidays_in_duration , enable_comp_off
EmployeePastLeaveRestrict	Specific exemptions or restrictions on applying	employee (FK), allow_past_leave , max_retroactive_days

Model Name	Purpose	Primary Attributes & Relationships
	for past-dated leave.	

6.3 Leave Application & Approval Workflow



- **Sandwich Rule Processing:** If configured on `LeaveTypeCondition`, weekends or public holidays occurring between leave days are automatically counted as leave consumption.
- **Accrual Engine:** Scheduled monthly background jobs credit fractional leave allocations (e.g. 1.75 days/month for Annual Leave) directly into `AvailableLeave`.

CHAPTER 7

Payroll, Compensation & Benefits Administration

(`payroll`)

Employment contracts, salary structures, earnings, statutory deductions, tax brackets, loans, and batch payslips.

7.1 Overview & Responsibilities

The `payroll` application constitutes the financial compensation core of HRMS. It models complex remuneration packages, computes earnings allowances, calculates statutory deductions and income tax brackets, processes employee loans and expense reimbursements, generates monthly batch payslips, renders official PDF payslips, and archives detailed audit trails.

7.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
Contract	Employment terms, contract validity, and compensation baseline.	employee (FK), contract_title, wage_type (Monthly/Hourly), wage, start_date, end_date, salary_structure (FK), status (Draft/Active/Terminated)
SalaryStructure	Master compensation template combining allowances and deduction rules.	structure_name, company (FK), base_salary_percentage, description
Allowance	Earnings components (e.g. Basic, HRA, Medical, Transport, Performance Bonus).	allowance_name, company (FK), value_type (Fixed Amount / Percentage of Base), value, is_taxable, is_condition_based
Deduction	Deduction components (e.g. Provident Fund, Health Insurance, Professional Tax).	deduction_name, company (FK), value_type (Fixed / Percentage), value, is_statutory
FilingStatus & TaxBracket	Tax slabs, progressive tax rates, and standard deduction brackets.	filing_status_name, min_income, max_income, tax_percentage, fixed_tax_amount
Payslip	Master monthly remuneration statement for an employee.	employee (FK), contract (FK), month, year, basic_pay, gross_pay, total_allowance, total_deduction, net_pay, status (Draft/Generated/Approved/Paid), pdf_file
PayslipAutoGenerate	Scheduled batch runner configuration for automated monthly payroll.	company (FK), generation_day_of_month, auto_send_email, is_active
LoanAccount	Corporate employee loans and automated installment recovery.	employee (FK), loan_amount, monthly_installment, total_installments, paid_installments, remaining_balance, interest_rate, status
Reimbursement	Employee business expense claims and receipts.	employee (FK), claim_title, amount, receipt_date, status (Requested/Approved/Paid), approved_by
ReimbursementFile	Scanned receipts, invoices, and expense documentation.	reimbursement (FK), attachment_file, uploaded_at
PayrollGeneralSetting	Global payroll parameters: currency, work days per month, loss-of-pay formula.	currency_symbol, default_work_days_per_month, lop_calculation_mode, payslip_prefix
EncashmentGeneralSettings	Policies for leave encashment calculations	encashable_leave_types (M2M), encashment_wage_basis (Basic vs Gross),

Model Name	Purpose	Primary Attributes & Relationships
	upon year-end or separation.	max_encashable_days

7.3 Salary Calculation Formula & Workflow

The calculation of monthly remuneration follows rigorous financial formulas:

- 1. **Working Days & Loss of Pay (LOP):**
$$\text{Effective Days} = \text{Total Month Days} - \text{Unpaid Leaves} - \text{Unauthorized Absences}$$
$$\text{Base Pro-rated Salary} = \left(\frac{\text{Contract Wage}}{\text{Work Days in Month}} \right) \times \text{Effective Days}$$
- 2. **Gross Earnings:**
$$\text{Gross Pay} = \text{Base Salary} + \sum \text{Allowances} + \text{Approved Overtime} + \text{Reimbursements}$$
- 3. **Total Deductions:**
$$\text{Total Deductions} = \sum \text{Statutory Deductions} + \text{Income Tax} + \text{Loan Installment} + \text{Disciplinary Fines}$$
- 4. **Net Disbursed Salary:**
$$\text{Net Pay} = \text{Gross Pay} - \text{Total Deductions}$$

When batch payslip generation executes, the system creates individual `Payslip` records, generates formatted PDF documents, stores them in the media archive, and optionally delivers them via email to each employee.

CHAPTER 8

Performance Management System (`pms`)

Objectives & Key Results (OKRs), KPIs, 360-degree feedback, appraisal review cycles, and gamified bonus points.

8.1 Overview & Responsibilities

The `pms` application facilitates modern organizational alignment through Objectives and Key Results (OKRs), annual and quarterly appraisal cycles, 360-degree peer reviews, 1-on-1 performance meetings, and gamified employee recognition points.

8.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
<code>Period</code>	Appraisal evaluation timeframes (e.g. Q1 2026, Annual Review 2026).	<code>period_name</code> , <code>company</code> (FK), <code>start_date</code> , <code>end_date</code> , <code>is_active</code>
<code>Objective</code>	High-level organizational, departmental, or team strategic goals.	<code>title</code> , <code>company</code> (FK), <code>department</code> (FK), <code>period</code> (FK), <code>target_value</code> , <code>unit</code> , <code>description</code>
<code>KeyResult</code>	Specific, quantifiable outcomes required to accomplish an Objective.	<code>objective</code> (FK), <code>title</code> , <code>start_value</code> , <code>target_value</code> , <code>current_value</code> , <code>unit</code> , <code>weightage_percentage</code>
<code>EmployeeObjective</code> & <code>EmployeeKeyResult</code>	Cascaded individual goals assigned to specific employees.	<code>employee</code> (FK), <code>objective</code> (FK), <code>progress_percentage</code> , <code>self_rating</code> , <code>manager_rating</code> , <code>status</code>
<code>QuestionTemplate</code> & <code>Question</code>	Standardized evaluation forms for appraisals and feedback surveys.	<code>template_name</code> , <code>question_text</code> , <code>question_type</code> (Rating 1-5, Text, Multiple Choice)
<code>Feedback</code> & <code>AnonymousFeedback</code>	Continuous peer praise, constructive feedback, and anonymous surveys.	<code>employee</code> (FK), <code>reviewer</code> (FK), <code>period</code> (FK), <code>feedback_text</code> , <code>rating</code> , <code>is_anonymous</code>
<code>Meetings</code> & <code>MeetingsAnswer</code>	1-on-1 performance review sessions and documented action items.	<code>employee</code> (FK), <code>manager</code> (FK), <code>meeting_date</code> , <code>agenda</code> , <code>minutes</code> , <code>next_steps</code>
<code>EmployeeBonusPoint</code> & <code>BonusPointSetting</code>	Gamified peer-to-peer and managerial recognition point ledger.	<code>employee</code> (FK), <code>points</code> , <code>awarded_by</code> (FK), <code>reason</code> , <code>monthly_award_limit</code>

8.3 Appraisal Review & OKR Tracking Process

1. **Goal Setting:** At the beginning of each `Period` , strategic `Objective` targets are established and broken down into quantifiable `KeyResult` milestones.

2. **Cascading:** Objectives cascade down to individual `EmployeeObjective` records. Employees update progress percentages weekly or monthly.
3. **Appraisal Review:** At period close, formal evaluations are initiated:
4. **Self-Assessment:** The employee provides self-ratings and narrative justifications.
5. **360-Degree Feedback:** Selected peers, direct reports, and cross-functional collaborators submit evaluations via `Feedback`.
6. **Manager Review:** The reporting manager reviews combined feedback, assigns final ratings, and conducts a formal `Meetings` review.

CHAPTER 9

Asset & Equipment Lifecycle Management (`asset`)

Hardware and software inventory, batch procurement, employee allocation, condition photos, and returns.

9.1 Overview & Responsibilities

The `asset` application tracks the complete custody and lifecycle of company physical and digital property—laptops, monitors, mobile phones, peripherals, software licenses, and vehicles. It manages procurement batches, barcode/serial number identification, employee allocations, handover agreements, return condition photographic proof, and maintenance reporting.

9.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
AssetCategory	Classification of asset types (e.g. Laptops, Mobile Devices, Furniture, Software).	category_name , company (FK), description , depreciation_rate_percentage
AssetLot	Procurement batches and purchase orders.	lot_number , vendor_name , purchase_date , invoice_number , total_quantity , total_cost
Asset	Master asset specification profile.	asset_name , category (FK), lot (FK), manufacturer , model_number , specifications
AssetItem	Distinct individual physical unit with unique tracking identifiers.	asset (FK), tracking_id , serial_number , barcode , purchase_cost , warranty_expiry_date , status (Available/Assigned/In Repair/Disposed)
AssetAssignment	Custody record tracking assignment of an asset to an employee.	asset_item (FK), assigned_to (FK to Employee), assigned_by (FK), assigned_date , expected_return_date , notes
AssetRequest	Employee self-service requisition for equipment or replacement.	employee (FK), category (FK), reason , urgency , status (Requested/Approved/Allocated/Rejected)
ReturnImages	Photographic documentation verifying asset physical condition upon return.	asset_assignment (FK), image_file , uploaded_at , notes
AssetReport	Damage reports, maintenance logs, and depreciation records.	asset_item (FK), report_type (Maintenance/Damage/Audit), cost , description , reported_by
AssetDocuments	Invoices, warranty cards, user manuals, and procurement contracts.	asset (FK), document_file , document_name , uploaded_at

9.3 Asset Lifecycle Workflow

1. **Procurement & Intake:** Assets arrive in bulk batches recorded under `AssetLot`. Individual physical items are registered in `AssetItem` with unique serial numbers, barcodes, and warranty dates.
2. **Allocation:** When an employee joins or requests equipment via `AssetRequest`, HR or IT assigns an available `AssetItem` creating an `AssetAssignment`. An official handover form is generated.
3. **Return & Condition Audit:** Upon offboarding or equipment upgrades, the asset is returned. IT captures digital photographs stored in `ReturnImages` to confirm device condition, screen integrity, and hardware completeness before releasing clearance.

CHAPTER 10

Recruitment & Applicant Tracking System (`recruitment`)

Job openings, multi-stage hiring pipelines, resume parsing, interview scorecards, and candidate skill zones.

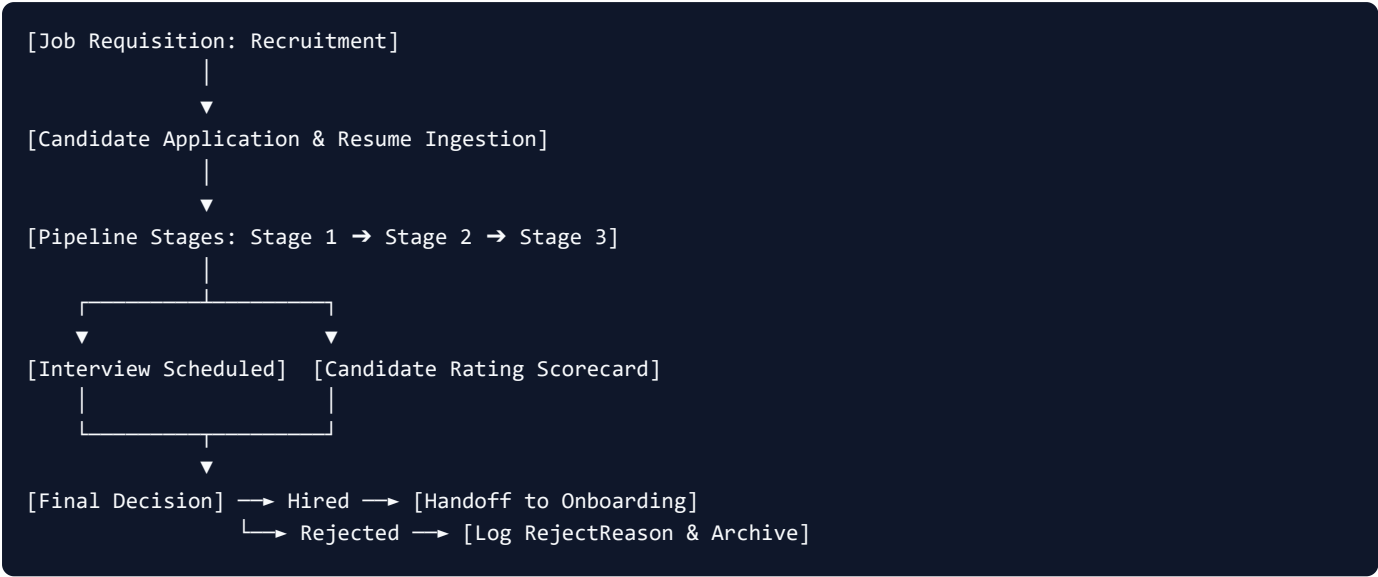
10.1 Overview & Responsibilities

The `recruitment` application delivers an enterprise Applicant Tracking System (ATS). It oversees talent acquisition from vacancy requisition, job posting, and candidate intake to resume parsing, pipeline stage transitions, interview scheduling, scorecard rating, and talent pool archiving.

10.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
Recruitment	Job opening campaign.	recruitment_title , company (FK), department (FK), job_position (FK), open_positions , start_date , end_date , job_description , status (Open/Closed/On Hold)
Stage	Configurable hiring pipeline stages for a recruitment campaign.	recruitment (FK), stage_name (e.g. Applied, Tech Screening, Panel Interview, Offer), sequence_order
Candidate	Applicant profile and recruitment progress tracker.	recruitment (FK), stage (FK), first_name , last_name , email , phone , current_ctc , expected_ctc , notice_period_days , status
Resume	Candidate resume documents and parsed plain-text representation.	candidate (FK), resume_file , parsed_text , uploaded_at
Skill & SkillZone	Skill taxonomy tags and talent pools for future requisition matching.	skill_name , category , candidates (M2M via SkillZoneCandidate)
InterviewSchedule	Scheduled interview rounds with panel links and calendar integrations.	candidate (FK), stage (FK), interview_date , interview_time , interviewers (M2M), meeting_link , status
CandidateRating	Interviewer scorecard evaluations, criteria ratings, and comments.	candidate (FK), interview (FK), interviewer (FK), rating (1-5), technical_skills , communication , recommendation
CandidateDocumentRequest & CandidateDocument	Pre-offer document requests (degrees, previous pay slips, ID proofs).	candidate (FK), document_title , file , status (Pending/Uploaded/Verified)
RejectReason & RejectedCandidate	Standardized rejection reasons and talent pool archiving.	candidate (FK), reject_reason (FK), rejection_notes , rejected_at
RecruitmentSurvey & SurveyTemplate	Candidate candidate-experience feedback surveys.	title , questions , answers (FK to RecruitmentSurveyAnswer)

10.3 Hiring Pipeline Execution



CHAPTER 11

Employee Onboarding Management (`onboarding`)

Pre-boarding portal, onboarding stages, task checklists, document submission, and conversion to employee.

11.1 Overview & Responsibilities

The `onboarding` application guides new hires from the moment an offer is accepted until their first 90 days of employment. It provides an external self-service portal where candidates can upload compliance documentation, complete onboarding checklists, and familiarize themselves with company culture prior to day one.

11.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
<code>OnboardingStage</code>	Sequential phases in the onboarding journey.	<code>stage_name</code> (e.g. Pre-boarding, Day 1 Orientation, Week 1 Induction, Probation), <code>sequence_order</code> , <code>company</code> (FK)
<code>OnboardingTask</code>	Reusable checklist tasks assigned to departments or the candidate.	<code>stage</code> (FK), <code>task_title</code> , <code>assigned_department</code> (IT/HR/Finance/Facilities), <code>due_days_from_joining</code> , <code>is_mandatory</code>
<code>OnboardingCandidate</code>	Prospective employee record in the onboarding pipeline.	<code>candidate</code> (FK from recruitment), <code>first_name</code> , <code>last_name</code> , <code>email</code> , <code>joining_date</code> , <code>status</code> (In Progress / Completed)
<code>CandidateStage</code>	Tracks a candidate's current stage completion status.	<code>candidate</code> (FK to <code>OnboardingCandidate</code>), <code>stage</code> (FK), <code>is_completed</code> , <code>completed_at</code>
<code>CandidateTask</code>	Specific task assignment for an onboarding candidate.	<code>candidate</code> (FK), <code>task</code> (FK to <code>OnboardingTask</code>), <code>status</code> (Pending/In Review/Completed), <code>submitted_file</code> , <code>verified_by</code>
<code>OnboardingPortal</code>	Security access tokens for external pre-hire portal access.	<code>candidate</code> (FK), <code>access_token</code> , <code>token_expiry</code> , <code>is_active</code>

11.3 Onboarding Lifecycle

- Candidate Handoff:** A hired candidate in `recruitment` is promoted to `OnboardingCandidate`.
- Pre-Boarding Access:** An `OnboardingPortal` token is generated and emailed to the candidate. Without requiring full system login, the candidate accesses their private portal to provide bank information, national identity cards, emergency contacts, and signed agreements.
- Cross-Departmental Tasks:** `CandidateTask` entries are automatically triggered for departmental teams:
- IT:** Configure email account, prepare laptop, allocate software licenses.
- Admin / Facilities:** Issue building access pass and desk allocation.
- HR:** Verify educational credentials, initiate background verification.
- Employee Profile Generation:** Upon completing onboarding stages, a single action creates the permanent `Employee` profile, `EmployeeWorkInformation`, and `HRMSUser` account.

CHAPTER 12

Employee Offboarding & Exit Clearance (`offboarding`)

Resignations, notice periods, multi-department clearances (IT, HR, Finance), exit interviews, and settlement.

12.1 Overview & Responsibilities

The `offboarding` application governs employee separations—voluntary resignations, contract expirations, or terminations. It ensures strict governance across asset recovery, financial settlements, non-disclosure compliance, system de-provisioning, and exit interview analytics.

12.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
<code>ResignationLetter</code>	Formal resignation application submitted by an employee.	<code>employee</code> (FK), <code>resignation_date</code> , <code>requested_last_working_day</code> , <code>reason</code> , <code>status</code> (Requested/Accepted/Withdrawn/Rejected)
<code>Offboarding</code>	Approved separation case governing notice period and clearance.	<code>employee</code> (FK), <code>notice_period_days</code> , <code>approved_last_working_day</code> , <code>exit_reason</code> (FK), <code>status</code> (In Progress/Completed)
<code>OffboardingStage</code>	Departmental clearance checkpoints (IT, Finance, HR, Admin).	<code>stage_name</code> , <code>sequence_order</code> , <code>responsible_department</code> (FK)
<code>OffboardingTask</code>	Specific clearance items required by departments.	<code>stage</code> (FK), <code>task_title</code> , <code>description</code> , <code>is_mandatory</code>
<code>EmployeeTask</code>	Individual task assignment and clearance status per exiting employee.	<code>offboarding_employee</code> (FK), <code>task</code> (FK), <code>status</code> (Pending/Cleared/Rejected), <code>cleared_by</code> (FK), <code>clearance_date</code>
<code>ExitReason</code>	Standardized separation categories for workforce attrition analytics.	<code>reason_title</code> , <code>category</code> (Career Growth, Compensation, Personal, Relocation)
<code>OffboardingNote</code>	Confidential HR exit interview transcripts and settlement notes.	<code>offboarding</code> (FK), <code>notes</code> , <code>created_by</code> (FK), <code>created_at</code>
<code>OffboardingGeneralSetting</code>	Policy parameters for notice period waivers, gratuity rules, and account locking.	<code>auto_lock_account_on_exit</code> , <code>default_notice_days</code> , <code>require_asset_clearance</code>

12.3 Multi-Department Clearance Workflow

1. **Submission & Acceptance:** Employee submits `ResignationLetter`. Reporting manager and HR review notice obligations and approve official `approved_last_working_day`.
2. **Clearance Checklists:** System spawns clearance tasks across departments:
3. **IT Department:** Verify return of laptop and peripherals, revoke VPN and corporate email, remove GitHub/Jira access.
4. **Finance Department:** Calculate remaining loan balances, travel expense claims, and compute Full & Final (F&F) settlement amount.
5. **HR Department:** Conduct exit interview survey, process leave encashment calculations, prepare Relieving and Experience letters.
6. **Facilities:** Retrieve physical building access cards and company vehicle keys.
7. **Archival:** Upon 100% task clearance, the `Employee` record is marked `is_active = False` and `HRMSUser` account is locked.

CHAPTER 13

Project Management & Timesheets (`project`)

Project tracking, Kanban task boards, milestones, and billable hour timesheets.

13.1 Overview & Responsibilities

The `project` application provides project tracking and workforce utilization tools directly inside HRMS. Teams can create projects, organize tasks across Kanban stages, assign employees, monitor deadlines, and log daily timesheets for payroll and client billing.

13.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
Project	Master project record.	<code>project_name</code> , <code>company</code> (FK), <code>client_name</code> , <code>project_manager</code> (FK to Employee), <code>start_date</code> , <code>end_date</code> , <code>budget</code> , <code>status</code> (Planning/Active/On Hold/Completed)
ProjectStage	Workflow states for project tasks (e.g. Backlog, In Progress, In Review, Done).	<code>project</code> (FK), <code>stage_name</code> , <code>sequence_order</code>
Task	Individual work assignments and deliverables.	<code>project</code> (FK), <code>stage</code> (FK), <code>task_name</code> , <code>assigned_to</code> (M2M to Employee), <code>priority</code> (Low/Medium/High/Urgent), <code>estimated_hours</code> , <code>due_date</code> , <code>status</code>
TimeSheet	Daily hours logged by employees against specific tasks and projects.	<code>employee</code> (FK), <code>project</code> (FK), <code>task</code> (FK), <code>date</code> , <code>hours_spent</code> , <code>is_billable</code> , <code>description</code> , <code>status</code> (Draft/Submitted/Approved)

CHAPTER 14

Helpdesk & Internal Support Ticketing (`helpdesk`)

Internal ticketing system, departmental routing, SLA timers, comments, and knowledge base FAQs.

14.1 Overview & Responsibilities

The `helpdesk` module provides an internal support desk where employees submit service requests, report hardware issues, query payroll discrepancies, or resolve policy questions. It includes SLA tracking, departmental routing, two-way threaded discussions, and a self-service FAQ knowledge base.

14.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
<code>TicketType</code>	Ticket classification (e.g. IT Incident, Payroll Query, HR Request, Facilities).	<code>type_name</code> , <code>department</code> (FK), <code>sla_hours</code> , <code>company</code> (FK)
<code>Ticket</code>	Individual support ticket.	<code>ticket_number</code> , <code>employee</code> (FK), <code>ticket_type</code> (FK), <code>title</code> , <code>description</code> , <code>priority</code> (Low/Medium/High/Urgent), <code>status</code> (Open/Assigned/In Progress/Resolved/Closed), <code>assigned_to</code> (FK to Employee)
<code>DepartmentManager</code>	Designates lead support agents responsible for ticket queues per department.	<code>department</code> (FK), <code>manager</code> (FK to Employee), <code>company</code> (FK)
<code>Comment</code>	Threaded discussions between employee and assigned support agents.	<code>ticket</code> (FK), <code>author</code> (FK to User), <code>comment_text</code> , <code>is_internal_note</code> , <code>created_at</code>
<code>Attachment</code>	File attachments (screenshots, logs, error reports) linked to tickets.	<code>ticket</code> (FK), <code>attachment_file</code> , <code>uploaded_at</code>
<code>ClaimRequest</code>	Specific financial or resource claims associated with a ticket.	<code>ticket</code> (FK), <code>claim_amount</code> , <code>claim_reason</code> , <code>status</code>
<code>FAQCategory</code> & <code>FAQ</code>	Self-service articles answering common employee questions.	<code>category</code> (FK), <code>question</code> , <code>answer_html</code> , <code>is_published</code>

CHAPTER 15

Reporting & Business Intelligence Engine (`report`)

Dynamic report builder, scheduled email subscriptions, saved views, and multi-format exports (CSV, Excel, PDF).

15.1 Overview & Responsibilities

The `report` application provides comprehensive cross-module analytics. HR directors and executive leadership can generate real-time metrics on headcount distribution, attrition trends, attendance punctuality, leave utilization, asset allocations, and payroll expenditures.

15.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
<code>ReportTemplate</code>	Pre-configured or custom query definitions for specific modules.	<code>template_name</code> , <code>module</code> (Employee/Attendance/Leave/Payroll/Asset), <code>selected_fields</code> , <code>filter_criteria</code>
<code>ReportSubscription</code>	Scheduled automated report delivery to managers' emails.	<code>template</code> (FK), <code>recipient_emails</code> , <code>frequency</code> (Daily/Weekly/Monthly), <code>export_format</code> (CSV/Excel/PDF), <code>is_active</code>
<code>ReportFavorite</code>	Personal pinned reports for quick access on user dashboards.	<code>user</code> (FK), <code>template</code> (FK)
<code>ReportSavedView</code>	User-customized column sets, sorting rules, and filter presets.	<code>template</code> (FK), <code>user</code> (FK), <code>view_name</code> , <code>column_configuration</code>
<code>ReportRunLog</code>	Audit log tracking report execution duration, record counts, and user identity.	<code>template</code> (FK), <code>executed_by</code> (FK), <code>executed_at</code> , <code>duration_seconds</code> , <code>rows_count</code>
<code>ReportAccess</code>	Role-based permission controls dictating who can run sensitive financial reports.	<code>template</code> (FK), <code>allowed_roles</code> (M2M), <code>allowed_departments</code> (M2M)

CHAPTER 16

RESTful API Architecture & Interactive Swagger UI (`hrms\_api`)

Django REST Framework endpoints, SimpleJWT bearer tokens, Swagger UI, ReDoc, and module serializers.

16.1 Overview & Architecture

The `hrms_api` application exposes secure RESTful endpoints for integration with mobile applications, corporate portals, and third-party systems. Built with Django REST Framework (DRF) and `drf_yasg`, it provides full OpenAPI 3.0 interactive documentation via Swagger UI (`/api/v1/swagger/`) and ReDoc (`/api/v1/redoc/`).

16.2 Authentication & Security

- **JWT Authentication:** Clients obtain JSON Web Token pairs via `/api/v1/auth/token/` and refresh via `/api/v1/auth/token/refresh/`.
 - **Bearer Token Header:** All subsequent API calls require: `http Authorization: Bearer <jwt_access_token>`
 - **Permission Classes:** Endpoints enforce `IsAuthenticated` alongside granular role checks. Managerial endpoints enforce custom permissions returning HTTP 403 Forbidden for unauthorized tiers.
-

16.3 Module API Coverage Matrix

The API encompasses 14 dedicated packages under `hrms_api/api_urls/`:

API Domain	Base Endpoint	Key Operations
Auth	/api/v1/auth/	Token obtain, token refresh, password reset, user profile
Base	/api/v1/base/	Companies, departments, job positions, shifts, work types
Employee	/api/v1/employee/	Employee CRUD, work info, bank details, emergency contacts
Attendance	/api/v1/attendance/	Clock-in, clock-out, attendance validation, overtime requests
Leave	/api/v1/leave/	Leave types, available balances, request submissions, approvals
Payroll	/api/v1/payroll/	Contracts, salary structures, payslip queries, expense claims
PMS	/api/v1/pms/	OKRs, objectives, key results, 360 feedback submissions
Asset	/api/v1/asset/	Asset catalog, asset items, assignment history, asset requests
Recruitment	/api/v1/recruitment/	Openings, candidates, interview schedules, candidate ratings
Onboarding	/api/v1/onboarding/	Onboarding candidates, task lists, portal submissions
Offboarding	/api/v1/offboarding/	Resignation requests, departmental clearance checklists
Project	/api/v1/project/	Projects, tasks, stages, timesheet logging
Helpdesk	/api/v1/helpdesk/	Tickets, comments, ticket types, attachments
Notifications	/api/v1/notifications/	Unread notifications, mark-as-read, user broadcasts

CHAPTER 17

Enterprise Authentication, RBAC & Security (`hrms\_auth`, `base`)

Custom user models, role-based access control (RBAC), two-factor authentication (2FA), and session protection.

17.1 Overview & Responsibilities

HRMS implements enterprise-grade identity and access governance via `hrms_auth`. It supports username or corporate email login, multi-company role assignments, TOTP two-factor authentication, forced password rotation, and active session protection.

17.2 Database Models & User Architecture

- `HRMSUser` : Primary authentication model extending `AbstractUser`. Attributes: `email` (unique index), `username`, `phone`, `is_verified`, `two_factor_enabled`, `force_password_change`, `company` (FK).
 - `LegacyUser` : Migration bridge preserving backward compatibility with legacy authentication systems.
 - `AuthUserGroups` & `AuthUserUserPermissions` : Scoped permission mappings associating users with granular roles within their specific operating company.
-

17.3 Security Defenses & Hardening

1. **Two-Factor Authentication (2FA)**: Generates standard TOTP secrets compatible with Google Authenticator or Microsoft Authenticator. Mandatory for payroll and administrative roles.
2. **Brute Force Protection**: Failed login tracking locks accounts after configurable attempts, logging the event in `hrms_audit.AccountBlockUnblock`.
3. **SVG & File Sanitization**: `SVGSecurityMiddleware` strips executable scripts from XML-based image files to eliminate SVG XSS vectors.
4. **Session Expiry & CSRF**: Secure, HTTP-only session cookies with strict SameSite attributes and mandatory CSRF token verification on all POST/PUT/DELETE requests.

CHAPTER 18

Audit Trails & Regulatory Compliance (`hrms\_audit`)

Complete changelog tracking, field-level diffs, audit tags, and user block/unblock logs via django-auditlog.

18.1 Overview & Responsibilities

The `hrms_audit` application provides complete regulatory compliance and non-repudiation tracking. Integrated with `django-auditlog` and `simple_history`, every data modification across sensitive entities (contracts, wages, employee records, attendance punches, and leave approvals) is archived with timestamps, user IDs, IP addresses, and before-and-after diffs.

18.2 Database Models Specification

Model Name	Purpose	Primary Attributes & Relationships
<code>HRMSAuditLog</code>	Master changelog recording entity state mutations.	<code>content_type</code> , <code>object_id</code> , <code>actor</code> (FK to User), <code>action</code> (Create/Update/Delete), <code>changes_json</code> , <code>timestamp</code> , <code>ip_address</code>
<code>HRMSAuditInfo</code>	Operational classification and context attached to audit entries.	<code>audit_log</code> (FK), <code>module_name</code> , <code>severity</code> (Low/Medium/High/Critical), <code>action_description</code>
<code>AuditTag</code>	Categorical tags (e.g. "Salary Revision", "Manual Attendance Punch", "Security Alert").	<code>tag_name</code> , <code>color_code</code>
<code>HistoryTrackingFields</code>	Dynamic configuration specifying which model fields trigger audit tracking.	<code>model_name</code> , <code>tracked_fields_list</code> , <code>is_active</code>
<code>AccountBlockUnblock</code>	Audit records of user accounts locked due to failed logins or admin actions.	<code>user</code> (FK), <code>action</code> (Blocked/Unblocked), <code>reason</code> , <code>action_by</code> (FK), <code>timestamp</code>

CHAPTER 19

Automations, Notifications & Scheduled Tasks (`hrms_automations``, `notifications``)

Event-driven email automations, background cron processing via APScheduler, and real-time in-app alerts.

19.1 Overview & Responsibilities

The `hrms_automations` and `notifications` modules eliminate manual overhead through event triggers, automated notifications, and scheduled background tasks.

19.2 Database Models & Features

- `MailAutomation` : Event-driven rules linking system triggers (e.g. On Employee Creation, Contract Expiration, Birthday, Work Anniversary) to dynamic HTML mail templates.
- `Notification` : In-app notification center recording alerts, approval requests, and announcements with unread counter badges.
- `django-apscheduler` : Background task scheduler executing recurring operations:
 - Daily attendance aggregation and hours compilation.
 - Nightly contract expiration scans with warning alerts to HR.
 - Scheduled automated monthly payroll batch runs.
 - Periodic database backups and cloud sync.

CHAPTER 20

Document Generation & Database Templates (`hrms\_documents`, `hrms\_dbtemplate`)

Dynamic rich-text letter templates, merge tags, document requests, and automated PDF letter generation.

20.1 Overview & Responsibilities

The `hrms_documents` and `hrms_dbtemplate` applications automate the authoring and issuance of corporate letters, agreements, and certificates.

20.2 Database Models & Features

- `Template` & `TemplateVersion` : Rich-text letter templates (Offer Letters, Experience Certificates, Non-Disclosure Agreements, Salary Certificates) with version control.
- **Dynamic Merge Variables:** Placeholders such as `{{employee_name}}`, `{{job_title}}`, `{{company_name}}`, and `{{salary_ctc}}` are interpolated with live database values at runtime.
- `DocumentRequest` : Employee self-service requests for formal letters, approval workflow, automated PDF generation, and archive downloads.

CHAPTER 21

Database Backup & Disaster Recovery (`hrms\_backup`, `pg\_backup`)

Automated and on-demand PostgreSQL database dumps, local encryption, and Google Drive cloud archives.

21.1 Overview & Responsibilities

The `hrms_backup` and `pg_backup` applications provide data protection and disaster recovery for PostgreSQL databases.

21.2 Features & Models

- `LocalBackup`: Automated or manual execution of PostgreSQL `pg_dump` snapshots stored in timestamped, encrypted local volumes.
- `GoogleDriveBackup`: Remote off-site backup synchronization utilizing Google Drive API service credentials.
- **One-Click Restoration**: Web-based database restoration interface with automated connection termination and integrity checks.

CHAPTER 22

External Enterprise Integrations (WhatsApp, LDAP, Outlook, Meet)

WhatsApp Business API messaging, Active Directory LDAP directory sync, Microsoft Outlook SSO, and Google Meet.

22.1 WhatsApp Business Integration (`whatsapp`)

- **Models:** `WhatsappCredentials` , `WhatsappFlowDetails` .
- **Functionality:** Integrates with the Meta WhatsApp Business API to dispatch instant transactional alerts (shift assignment notices, interview confirmations, leave request updates).

22.2 Enterprise Directory / LDAP Synchronization (`hrms_ldap`)

- **Models:** `LDAPSettings` .
- **Functionality:** Binds to corporate Active Directory or OpenLDAP servers for centralized authentication, user account provisioning, and group-to-role mappings.

22.3 Microsoft Outlook OAuth Single Sign-On (`outlook_auth`)

- **Models:** `AzureApi` .
- **Functionality:** Implements Microsoft Entra ID (Azure AD) OAuth 2.0 Single Sign-On, enabling enterprise users to authenticate with corporate Microsoft 365 accounts.

22.4 HRMS Meet Video Conferencing (`hrms_meet`)

- **Models:** `GoogleCloudCredential` , `GoogleCredential` , `GoogleMeeting` .
- **Functionality:** Integrates with Google Workspace APIs to generate Google Meet interview video rooms automatically upon interview scheduling.

CHAPTER 23

UI/UX, Dynamic Themes, Guided Tours & Accessibility

White-label theme engine, WCAG accessibility controls, interactive product tours, dynamic fields, and widgets.

23.1 Dynamic Theme Engine (`hrms_theme`)

- **Models:** `HRMSTheme`, `CompanyTheme`.
- **Functionality:** Dynamic white-label branding allowing each corporate tenant to upload logos, select primary/secondary brand palettes, and style the navigation sidebar.

23.2 WCAG Accessibility (`accessibility`)

- **Models:** `DefaultAccessibility`.
- **Functionality:** Built-in accessibility toolset offering high-contrast color modes, text resizing, and OpenDyslexic font toggles for inclusive usability.

23.3 Interactive Guided Tours (`hrms_tour`)

- **Models:** `Tour`, `TourStep`, `TourProgress`.
- **Functionality:** Interactive, step-by-step walkthroughs powered by `tourController.js` that guide newly provisioned employees and managers through features.

23.4 Dynamic Fields Engine (`dynamic_fields`)

- **Models:** `Choice`, `DynamicField`.
- **Functionality:** Allows administrators to attach custom attributes (text, number, date, dropdowns) to core models at runtime without writing migrations.

23.5 Dynamic Views & Widgets (`hrms_views`, `hrms_widgets`, `hrms_crums`)

- Configurable table columns, drag-and-drop column orders, saved search filters, customizable dashboard widgets, and breadcrumb trails.

CHAPTER 24

DevOps, Containerization & Production Deployment

Docker multi-stage builds, Docker Compose, Nginx reverse proxy, PostgreSQL, health probes, and CI/CD.

24.1 Container Architecture & Multi-Stage Dockerfile

The HRMS container build leverages a hardened, multi-stage Debian-based Python 3.12 environment with essential system libraries for OpenCV (face detection), WeasyPrint (PDF generation), PostgreSQL client libraries, and LDAP development tools:

```
# Multi-stage production build summary
FROM python:3.12-slim AS builder
RUN apt-get update && apt-get install -y --no-install-recommends      build-essential libpq-dev libldap2-dev l
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

24.2 Docker Compose Configuration

1. Development Stack (`docker-compose.yml`)

- Binds project source code directly into `/app/` for live hot-reloading.
- Runs PostgreSQL 16 on port 5432 and Redis 7 on port 6379.
- Exposes web service at `http://127.0.0.1:8080`.

2. Production Stack (`docker-compose.prod.yml`)

- Deploys Nginx as the front-facing reverse proxy handling TLS and static file delivery.
- Runs Gunicorn with multiple worker processes.
- Mounts persistent Docker volumes (`postgres_data` , `redis_data` , `media_volume`).

24.3 Health Probes & Monitoring

The system exposes two dedicated endpoints in `hrms/urls.py`: - `/health/`: Cheap liveness probe verifying that the application process is alive (used by Docker HEALTHCHECK). - `/ready/`: Deep readiness probe verifying active connectivity to PostgreSQL and Redis cache read/write capability before routing traffic.

24.4 Continuous Integration & GitHub Actions

Automated CI/CD workflows under `.github/workflows/` execute on every push: 1. **Linting & Code Style**: Enforces PEP 8 standards with Flake8 and Black. 2. **Unit & Integration Tests**: Runs Django test suites across core applications. 3. **Container Build Verification**: Verifies clean Docker container packaging and health probe responses.